

Jacobi và Gauss-Seidel cho phương trình Laplace/Poisson 2D

1. Bài toán elliptic

Trong nhiều bài toán thế tĩnh điện, ta cần giải phương trình Laplace hoặc Poisson:

Phương trình Laplace

$$\nabla^2 u(x, y) = 0$$

hay

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

Đây là phương trình elliptic.

Phương trình Poisson

$$\nabla^2 u(x, y) = f(x, y)$$

Trong tĩnh điện, thường gặp dạng:

$$\nabla^2 V = -\frac{\rho}{\epsilon_0}$$

hoặc trong hệ đơn vị khác:

$$\nabla^2 V = -4\pi\rho$$

2. Sai phân hữu hạn cho Laplace 2D

Chia miền tính toán thành lưới đều:

$$x_i = x_0 + ih$$

$$y_j = y_0 + jh$$

Với $\Delta x = \Delta y = h$, ta có xấp xỉ:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}$$

Thay vào phương trình Laplace:

$$\nabla^2 u = 0$$

ta được:

$$u_{i,j} = \frac{1}{4} (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})$$

Nghĩa là giá trị tại một điểm bằng trung bình cộng của 4 điểm lân cận.

3. Ý tưởng Jacobi

Jacobi dùng toàn bộ giá trị cũ để tính giá trị mới.

$$u_{i,j}^{(k+1)} = \frac{1}{4} (u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)})$$

Đặc điểm:

- Dễ hiểu.
 - Dễ vector hóa bằng NumPy.
 - Nhưng hội tụ chậm.
 - Tại mỗi vòng lặp phải tạo mảng mới hoặc copy mảng cũ.
-

4. Ý tưởng Gauss-Seidel

Gauss-Seidel dùng ngay giá trị mới vừa tính được.

$$u_{i,j}^{(k+1)} = \frac{1}{4} (u_{i+1,j}^{(k)} + u_{i-1,j}^{(k+1)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k+1)})$$

Đặc điểm:

- Hội tụ nhanh hơn Jacobi.
 - Cập nhật trực tiếp trên cùng một mảng.
 - Khó vector hóa hơn Jacobi.
 - Phù hợp để viết bằng vòng lặp `for`.
-

5. Solver tổng quát cho Laplace 2D

5.1. Import thư viện

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

plt.style.use("sci.mplstyle")
```

5.2. Hàm tạo điều kiện biên hình vuông

Quy ước:

```
u[i, 0]      = bottom
u[i, n-1]    = top
u[0, j]      = left
u[n-1, j]    = right
```

```
def tao_bien_hinhvuong(n, u_bottom, u_left, u_right, u_top):
    u = np.zeros((n, n), dtype=float)

    u[:, 0] = u_bottom
    u[:, -1] = u_top
    u[0, :] = u_left
    u[-1, :] = u_right

    return u
```

5.3. Hàm lưu kết quả ra file

```
def luu_file(filename, u, err, L, k):
    n = u.shape[0]

    with open(filename, "w", encoding="utf-8") as file:
        file.write(f"### Ket qua tai vong lap {k}\n")
        file.write(f"### Max error = {np.max(err):.6e}\n")
        file.write("#" * 80 + "\n")
        file.write(f"### {'x':>15s} {'y':>15s} {'V':>15s} {'Error':>15s}\n")

        for i in range(n):
            for j in range(n):
                x = i * L / (n - 1)
                y = j * L / (n - 1)
                file.write(f"{x:15.6f} {y:15.6f} {u[i,j]:15.6f} {err[i,j]:15.6e}\n")
            file.write("\n")
```

6. Jacobi solver

```
def solve_laplace_jacobi(u, L, N_max=100000, err_max=1e-6,
filename="jacobi_result.txt"):
    n = u.shape[0]

    for k in range(N_max):
        u_old = u.copy()

        u_new = u.copy()

        u_new[1:-1, 1:-1] = 0.25 * (
            u_old[2:, 1:-1]
            + u_old[:-2, 1:-1]
            + u_old[1:-1, 2:]
            + u_old[1:-1, :-2]
        )

        u = u_new

        err = np.abs(u - u_old)
        max_err = np.max(err)

        if max_err < err_max:
            print(f"Jacobi hoi tu sau {k} vong lap, max_err = {max_err:.3e}")
            luu_file(filename, u, err, L, k)
            return u

    print(f"Jacobi khong hoi tu sau {N_max} vong lap, max_err = {max_err:.3e}")
    luu_file(filename, u, err, L, N_max)
    return u
```

7. Gauss-Seidel solver

```
def solve_laplace_gauss_seidel(u, L, N_max=100000, err_max=1e-6,
filename="gs_result.txt"):
    n = u.shape[0]

    for k in range(N_max):
        u_old = u.copy()

        for i in range(1, n - 1):
            for j in range(1, n - 1):
                u[i, j] = 0.25 * (
                    u[i + 1, j]
```

```

        + u[i - 1, j]
        + u[i, j + 1]
        + u[i, j - 1]
    )

    err = np.abs(u - u_old)
    max_err = np.max(err)

    if max_err < err_max:
        print(f"Gauss-Seidel hoi tu sau {k} vong lap, max_err =
{max_err:.3e}")
        luu_file(filename, u, err, L, k)
        return u

    print(f"Gauss-Seidel khong hoi tu sau {N_max} vong lap, max_err =
{max_err:.3e}")
    luu_file(filename, u, err, L, N_max)
    return u

```

8. Hàm vẽ kết quả

8.1. Vẽ contour/heatmap

```

def plot_contour(u, L, filename="contour.png", title="The tinh dien"):
    n = u.shape[0]

    x = np.linspace(0, L, n)
    y = np.linspace(0, L, n)

    X, Y = np.meshgrid(x, y, indexing="ij")

    plt.figure(figsize=(8, 7))
    plt.contourf(X, Y, u, levels=100, cmap="coolwarm")
    plt.colorbar(label="V")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(title)
    plt.savefig(filename, dpi=300)
    plt.show()

```

8.2. Vẽ mặt 3D

```

def plot_surface(u, L, filename="surface.png", title="The tinh dien 3D"):
    n = u.shape[0]

```

```

x = np.linspace(0, L, n)
y = np.linspace(0, L, n)

X, Y = np.meshgrid(x, y, indexing="ij")

fig = plt.figure(figsize=(9, 9))
ax = fig.add_subplot(111, projection="3d")

ax.plot_surface(X, Y, u, color="white", edgecolor="gray", linewidth=0.3)

ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("V")
ax.set_title(title)

ax.view_init(elev=20, azim=60)

plt.savefig(filename, dpi=300)
plt.show()

```

9. Bài mẫu 1: Hộp vuông nổi đất, cạnh trên 100 V

Mô tả bài toán

Miền là hình vuông kích thước $L \times L$.

Điều kiện biên:

$$V(x, 0) = 0$$

$$V(0, y) = 0$$

$$V(L, y) = 0$$

$$V(x, L) = 100$$

Code

```

L = 100
n = 100

N_max = 100000
err_max = 1e-6

u = tao_bien_hinhvuong(
    n=n,
    u_bottom=0,

```

```

        u_left=0,
        u_right=0,
        u_top=100
    )

    u_result = solve_laplace_gauss_seidel(
        u=u,
        L=L,
        N_max=N_max,
        err_max=err_max,
        filename="bai1_gauss_seidel.txt"
    )

    plot_contour(
        u_result,
        L,
        filename="bai1_contour.png",
        title="Bai 1: The tinh dien"
    )

    plot_surface(
        u_result,
        L,
        filename="bai1_surface.png",
        title="Bai 1: The tinh dien 3D"
    )

```

10. Bài mẫu 2: Hai thanh điện cực bên trong hộp nối đất

Mô tả bài toán

Biên ngoài nối đất:

$$V = 0$$

Bên trong có hai thanh điện cực:

$$V = +100$$

và

$$V = -100$$

Hàm tạo điều kiện biên bài 2

```
def tao_bien_hai_thanh(n, L, u_bien=0, u_thap=-100, u_cao=100, chieudai=50,
    khoangcach=30):
    u = np.zeros((n, n), dtype=float)

    u[:, 0] = u_bien
    u[:, -1] = u_bien
    u[0, :] = u_bien
    u[-1, :] = u_bien

    n_chieudai = int(chieudai / L * (n - 1))
    n_khoangcach = int(khoangcach / L * (n - 1))

    center = n // 2

    j_tren = center + n_khoangcach // 2
    j_duoi = center - n_khoangcach // 2

    i_start = (n - n_chieudai) // 2
    i_end = i_start + n_chieudai

    u[i_start:i_end, j_tren] = u_cao
    u[i_start:i_end, j_duoi] = u_thap

    fixed_mask = np.zeros_like(u, dtype=bool)

    fixed_mask[:, 0] = True
    fixed_mask[:, -1] = True
    fixed_mask[0, :] = True
    fixed_mask[-1, :] = True

    fixed_mask[i_start:i_end, j_tren] = True
    fixed_mask[i_start:i_end, j_duoi] = True

    return u, fixed_mask
```

Gauss-Seidel solver có mask điểm cố định

Dùng cho bài có điện cực bên trong.

```
def solve_laplace_gauss_seidel_mask(u, fixed_mask, L, N_max=100000,
    err_max=1e-6, filename="gs_mask_result.txt"):
    n = u.shape[0]

    for k in range(N_max):
        u_old = u.copy()
```



```

for i in range(1, n - 1):
    for j in range(1, n - 1):

        if fixed_mask[i, j]:
            continue

        u[i, j] = 0.25 * (
            u[i + 1, j]
            + u[i - 1, j]
            + u[i, j + 1]
            + u[i, j - 1]
        )

    err = np.abs(u - u_old)
    max_err = np.max(err)

    if max_err < err_max:
        print(f"Gauss-Seidel hoi tu sau {k} vong lap, max_err =
{max_err:.3e}")
        luu_file(filename, u, err, L, k)
        return u

    print(f"Gauss-Seidel khong hoi tu sau {N_max} vong lap, max_err =
{max_err:.3e}")
    luu_file(filename, u, err, L, N_max)
    return u

```

Code giải bài 2

```

L = 100
n = 100

N_max = 100000
err_max = 1e-6

u, fixed_mask = tao_bien_hai_thanh(
    n=n,
    L=L,
    u_bien=0,
    u_thap=-100,
    u_cao=100,
    chieudai=50,
    khoangcach=30
)

u_result = solve_laplace_gauss_seidel_mask(
    u=u,

```

```

        fixed_mask=fixed_mask,
        L=L,
        N_max=N_max,
        err_max=err_max,
        filename="bai2_gauss_seidel.txt"
    )

    plot_contour(
        u_result,
        L,
        filename="bai2_contour.png",
        title="Bai 2: Hai thanh dien cuc"
    )

    plot_surface(
        u_result,
        L,
        filename="bai2_surface.png",
        title="Bai 2: Hai thanh dien cuc 3D"
    )

```

11. Bài mẫu 3: Phương trình Poisson 2D

Phương trình

$$\nabla^2 u = f(x, y)$$

Sai phân hữu hạn:

$$u_{i,j} = \frac{1}{4} (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - h^2 f_{i,j})$$

Lưu ý dấu của f phụ thuộc vào cách viết phương trình.

Nếu viết:

$$\nabla^2 u = f$$

thì dùng:

$$-h^2 f$$

Nếu viết:

$$\nabla^2 u = -\rho$$

thì dùng:

$$+h^2 \rho$$

Poisson solver

```
def solve_poisson_gauss_seidel(u, f, L, N_max=100000, err_max=1e-6,
    filename="poisson_result.txt"):
    n = u.shape[0]
    h = L / (n - 1)

    for k in range(N_max):
        u_old = u.copy()

        for i in range(1, n - 1):
            for j in range(1, n - 1):
                u[i, j] = 0.25 * (
                    u[i + 1, j]
                    + u[i - 1, j]
                    + u[i, j + 1]
                    + u[i, j - 1]
                    - h**2 * f[i, j]
                )

        err = np.abs(u - u_old)
        max_err = np.max(err)

        if max_err < err_max:
            print(f"Poisson GS hoi tu sau {k} vong lap, max_err = {max_err:.3e}")
            luu_file(filename, u, err, L, k)
            return u

        print(f"Poisson GS khong hoi tu sau {N_max} vong lap, max_err = {max_err:.3e}")
        luu_file(filename, u, err, L, N_max)
        return u
```

12. Bài mẫu 4: Nguồn Gaussian ở giữa hộp

Mô tả

Giải:

$$\nabla^2 u = -\rho(x, y)$$

với nguồn Gaussian ở giữa hộp:

$$\rho(x, y) = \rho_0 \exp \left[-\frac{(x - L/2)^2 + (y - L/2)^2}{2\sigma^2} \right]$$

Do phương trình là:

$$\nabla^2 u = -\rho$$

nên trong code Poisson ta đặt:

$$f = -\rho$$

Code

```
L = 100
n = 100

N_max = 100000
err_max = 1e-6

u = tao_bien_hinhvuong(
    n=n,
    u_bottom=0,
    u_left=0,
    u_right=0,
    u_top=0
)

x = np.linspace(0, L, n)
y = np.linspace(0, L, n)

X, Y = np.meshgrid(x, y, indexing="ij")

rho0 = 1.0
sigma = 5.0

rho = rho0 * np.exp(-((X - L/2)**2 + (Y - L/2)**2) / (2 * sigma**2))

f = -rho

u_result = solve_poisson_gauss_seidel(
    u=u,
    f=f,
    L=L,
    N_max=N_max,
    err_max=err_max,
    filename="bai3_poisson_gaussian.txt"
)

plot_contour(
    u_result,
    L,
    filename="bai3_poisson_gaussian_contour.png",
```

```

        title="Poisson: Nguon Gaussian"
    )

    plot_surface(
        u_result,
        L,
        filename="bai3_poisson_gaussian_surface.png",
        title="Poisson: Nguon Gaussian 3D"
    )

```

13. Đọc lại file kết quả và vẽ

Nếu đã lưu file dạng:

```
x y V Error
```

có thể đọc lại bằng:

```

x, y, V, err = np.loadtxt("bai1_gauss_seidel.txt", comments="#", unpack=True)

n = int(np.sqrt(len(x)))

X = x.reshape(n, n)
Y = y.reshape(n, n)
V2D = V.reshape(n, n)

plt.figure(figsize=(8, 7))
plt.contourf(X, Y, V2D, levels=100, cmap="coolwarm")
plt.colorbar(label="V")
plt.xlabel("x")
plt.ylabel("y")
plt.title("The tính diện")
plt.savefig("plot_from_file.png", dpi=300)
plt.show()

```

14. So sánh Jacobi và Gauss-Seidel

Phương pháp	Cách cập nhật	Tốc độ hội tụ	Dễ vector hóa NumPy
Jacobi	Dùng toàn bộ giá trị cũ	Chậm hơn	Dễ
Gauss-Seidel	Dùng ngay giá trị mới	Nhanh hơn	Khó hơn
SOR	Gauss-Seidel có hệ số thư giãn	Nhanh hơn nhiều	Trung bình

15. Lưu ý quan trọng khi làm bài

15.1. Dừng nhằm Jacobi với Gauss-Seidel

Code kiểu này là Jacobi:

```
u_old = u.copy()

u[1:-1, 1:-1] = 0.25 * (
    u_old[2:, 1:-1]
    + u_old[:-2, 1:-1]
    + u_old[1:-1, 2:]
    + u_old[1:-1, :-2]
)
```

Vì toàn bộ vế phải dùng `u_old`.

Code kiểu này là Gauss-Seidel:

```
for i in range(1, n - 1):
    for j in range(1, n - 1):
        u[i, j] = 0.25 * (
            u[i + 1, j]
            + u[i - 1, j]
            + u[i, j + 1]
            + u[i, j - 1]
        )
```

Vì `u[i-1, j]` và `u[i, j-1]` đã được cập nhật trong vòng lặp hiện tại.

15.2. Điều kiện hội tụ

Thường dùng:

$$\max |u^{(k+1)} - u^{(k)}| < \varepsilon$$

Trong code:

```
err = np.abs(u - u_old)
max_err = np.max(err)

if max_err < err_max:
    break
```

15.3. Với bài có điện cực bên trong

Phải giữ điện cực cố định.

Không được update các điểm thuộc điện cực.

Dùng `fixed_mask` là cách tổng quát nhất:

```
if fixed_mask[i, j]:  
    continue
```

16. Template thay code vào là chạy

```
import numpy as np  
import matplotlib.pyplot as plt  
  
plt.style.use("sci.mplstyle")  
  
def tao_bien_hinhvuong(n, u_bottom, u_left, u_right, u_top):  
    u = np.zeros((n, n), dtype=float)  
  
    u[:, 0] = u_bottom  
    u[:, -1] = u_top  
    u[0, :] = u_left  
    u[-1, :] = u_right  
  
    return u  
  
def luu_file(filename, u, err, L, k):  
    n = u.shape[0]  
  
    with open(filename, "w", encoding="utf-8") as file:  
        file.write(f"### Ket qua tai vong lap {k}\n")  
        file.write(f"### Max error = {np.max(err):.6e}\n")  
        file.write("#" * 80 + "\n")  
        file.write(f"### {'x':>15s} {'y':>15s} {'V':>15s} {'Error':>15s}\n")  
  
        for i in range(n):  
            for j in range(n):  
                x = i * L / (n - 1)  
                y = j * L / (n - 1)  
                file.write(f"{x:15.6f} {y:15.6f} {u[i,j]:15.6f} {err[i,j]:  
15.6e}\n")
```

```

        file.write("\n")

def solve_laplace_gauss_seidel(u, L, N_max=100000, err_max=1e-6,
    filename="result.txt"):
    n = u.shape[0]

    for k in range(N_max):
        u_old = u.copy()

        for i in range(1, n - 1):
            for j in range(1, n - 1):
                u[i, j] = 0.25 * (
                    u[i + 1, j]
                    + u[i - 1, j]
                    + u[i, j + 1]
                    + u[i, j - 1]
                )

        err = np.abs(u - u_old)
        max_err = np.max(err)

        if max_err < err_max:
            print(f"Hoi tu sau {k} vong lap, max_err = {max_err:.3e}")
            luu_file(filename, u, err, L, k)
            return u

    print(f"Khong hoi tu sau {N_max} vong lap, max_err = {max_err:.3e}")
    luu_file(filename, u, err, L, N_max)
    return u

def plot_contour(u, L, filename="contour.png", title="The tinh dien"):
    n = u.shape[0]

    x = np.linspace(0, L, n)
    y = np.linspace(0, L, n)

    X, Y = np.meshgrid(x, y, indexing="ij")

    plt.figure(figsize=(8, 7))
    plt.contourf(X, Y, u, levels=100, cmap="coolwarm")
    plt.colorbar(label="V")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(title)
    plt.savefig(filename, dpi=300)
    plt.show()

def plot_surface(u, L, filename="surface.png", title="The tinh dien 3D"):

```



```

n = u.shape[0]

x = np.linspace(0, L, n)
y = np.linspace(0, L, n)

X, Y = np.meshgrid(x, y, indexing="ij")

fig = plt.figure(figsize=(9, 9))
ax = fig.add_subplot(111, projection="3d")

ax.plot_surface(X, Y, u, color="white", edgecolor="gray", linewidth=0.3)

ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("V")
ax.set_title(title)

ax.view_init(elev=20, azim=60)

plt.savefig(filename, dpi=300)
plt.show()

# =====
# Main code
# =====

L = 100
n = 100

N_max = 100000
err_max = 1e-6

u = tao_bien_hinhvuong(
    n=n,
    u_bottom=0,
    u_left=0,
    u_right=0,
    u_top=100
)

u_result = solve_laplace_gauss_seidel(
    u=u,
    L=L,
    N_max=N_max,
    err_max=err_max,
    filename="laplace_result.txt"
)

plot_contour(
    u_result,

```

```
L,  
filename="laplace_contour.png",  
title="Laplace equation"  
)  
  
plot_surface(  
    u_result,  
    L,  
    filename="laplace_surface.png",  
    title="Laplace equation 3D"  
)
```